

Reinforcement Learning

Hyoung Joon, Kim

August 13, 2026

Contents

1	Introduction	5
I	Common Theories	6
2	Basic Concepts	7
2.1	Introduction	7
2.2	State, Action, Policy and Reward	7
2.2.1	State	7
2.2.2	Action	7
2.2.3	Policy	8
2.2.4	Reward	8
2.3	Trajectories, Returns and Episodes	8
2.3.1	Trajectory	8
2.3.2	Returns	8
2.3.3	Episodes	9
2.4	Markov Decision Process	9
2.4.1	Definition	9
3	Bellman Operator	10
3.1	Introduction	10
3.2	The State Value Function and Action Value Function	10
3.3	Generalized Bellman Equation	11
3.4	Bellman Equation	11
3.4.1	Matrix-Vector form of Bellman Equation	12
3.5	Bellman Optimality Equation	13
3.5.1	Optimal Policy	13
3.5.2	Bellman Optimality Equation	14
3.6	Solution of The Bellman Equations	14
3.6.1	Contraction Mapping Theorem	14
3.6.2	Existence and Uniqueness of the Solution of Bellman Equations	16
3.7	Optimal Policy, Optimal State Value and Bellman Optimality Equation	17
3.8	Factors that Influence Optimal Policies	17
3.8.1	Reward	18
3.8.2	Discount Rate	18
3.8.3	Invariance of the Optimal Policy	18

4	Stochastic Approximation Theory	19
4.1	Introduction	19
4.2	Prerequisite	19
4.3	Robbins-Monro Algorithm	19
4.4	Dvoretzky's Convergence Theorem	21
4.4.1	Extension of Dvoretzky's Theorem	22
4.5	Stochastic Gradient Descent	22
II	Algorithms	23
5	Dynamic Programming	24
5.1	Introduction	24
5.2	Value Iteration	24
5.2.1	Main Idea	24
5.2.2	Pseudocode	25
5.2.3	Convergence	25
5.3	Policy Iteration	25
5.3.1	Main Idea	25
5.3.2	Pseudocode	26
5.3.3	Convergence	26
5.4	Truncated Policy Iteration	26
5.4.1	Main Idea	26
5.4.2	Pseudocode	27
5.4.3	Convergence	27
6	Monte Carlo Methods	28
6.1	Introduction	28
6.2	Monte Carlo Basic	28
6.2.1	Main Idea	28
6.2.2	Pseudocode	29
6.2.3	Convergence	29
6.2.4	Pros and Cons	29
6.3	Monte Carlo with Exploring Starts	29
6.3.1	Main Idea	29
6.3.2	Pseudocode	30
6.3.3	Implementation Details	30
6.3.4	Pros and Cons	30
6.4	Monte Carlo without Exploring Starts	31
6.4.1	Main Idea	31
6.4.2	Pseudocode	32
6.4.3	Implementation Details	32
6.4.4	Pros and Cons	33
7	Temporal Difference Methods	34
7.1	Introduction	34

7.2	TD Learning of State Values	34
7.2.1	Main Idea	34
7.3	Sarsa	36
7.3.1	Main Idea	36
7.3.2	Pseudocode	37
7.3.3	Implementation Detail	37
7.3.4	Convergence	37
7.3.5	Pros and Cons	38
7.4	Q-Learning	38
7.4.1	Main Idea	38
7.4.2	Pseudocode	39
7.4.3	Implementation Detail	39
7.4.4	Convergence	39
7.4.5	Pros and Cons	39
7.5	Expected Sarsa	40
7.5.1	Main Idea	40
8	Function Approximation Methods	41
8.1	Introduction	41
8.2	TD Learning of State Values based on Function Approximation	41
8.2.1	Objective Function	41
8.3	Deep Q-Learning	42
8.3.1	Main Idea	42
8.3.2	Objective Function	42
8.3.3	Pseudocode	43
8.3.4	Implementation Details	43
8.3.5	Pros and Cons	43
9	Policy-based Methods	44
9.1	Introduction	44
9.2	Metrics for Defining Optimal Policies	44
9.2.1	Average State Value	44
9.2.2	Average Reward	45
9.3	Policy Gradient Theorem	45
9.4	REINFORCE	46
9.4.1	Main Idea	46
9.4.2	Objective Function	46
9.4.3	Pseudocode	47
9.4.4	Implementation Notes	47
9.4.5	Pros and Cons	47
10	Actor-Critic Methods	48
10.1	Introduction	48
10.2	Advantage Actor-Critic (A2C)	49
10.2.1	Main Idea	49
10.2.2	Pseudocode	50

10.2.3 Implementation Details	50
10.2.4 Pros and Cons	51
11 Model-based Methods	52
12 Comparisons of Algorithms	53
III Appendices	54
13 Probability Theory	55
13.1 Introduction	55
13.2 Measure-Theoretic Probability	55
13.2.1 Basic Definitions	55
14 Linear Algebra	57
14.1 Introduction	57
14.2 Geshgorin Circle Theorem	57

Chapter 1

Introduction

This document is a set of notes of algorithms of reinforcement learning and theories of reinforcement learning. Following books were mainly used for studying algorithms and theories.

- [\[Sut20\]](#);
- [\[Gra20\]](#);
- [\[Zha25\]](#);

Part I

Common Theories

Chapter 2

Basic Concepts

2.1 Introduction

In this section, we discuss common concepts that are used frequently in reinforcement learning.

2.2 State, Action, Policy and Reward

2.2.1 State

The **state** is the status of the agent with respect to the environment. The set of all possible states, the **state space**, is denoted as $\mathcal{S}$.

2.2.2 Action

The **action** is what agent can do, when a state is given. By choosing the action and taking it, the agent moves to the next state, which is called **state transition**. The set of all possible actions in state s , **action space** for state s , is denoted as $\mathcal{A}(s)$.

2.2.2.1 Transition Probability

Note that the transition can be stochastic. That is, the same choice of action in the same state may not lead to equivalent next state. We denote this probability with conditional probability as following:

$$p(s'|s, a) \tag{2.1}$$

where $s', s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

2.2.3 Policy

The **policy** tells the agent which actions to take at every state. The policy can be deterministic or stochastic. We denote the probability of taking action a under state s following policy π as $\pi(a|s)$.

2.2.4 Reward

Reward is one of the most unique concept of reinforcement learning. After the execution of action, the agent receives a reward, denoted as r . The agent's goal is to maximize the total reward that the agent receives. Due to this goal, the reward can encourage or discourage the agent's choice of selecting an action under certain state. Again, the reward can be stochastic, too.

One question that follows naturally is: Can't we just pick the actions that gives the most reward at every state? If this was correct, the reinforcement learning may not have existed. Since the reward is just an immediate information of the act, choosing only the action with highest reward is not the correct answer, most of the time. There can be a better choice in the long run.

2.3 Trajectories, Returns and Episodes

2.3.1 Trajectory

The **trajectory** is the footsteps of the agent. More formally, suppose that the agent is on the initial state s_0 and took action a_0 . The agent receives reward r_1 from environment and moves to state s_1 . Again, the agent take an action a_1 , receive reward r_1 and move to state s_2 . Repeating this, we get the trajectory of an agent:

$$s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T. \quad (2.2)$$

2.3.2 Returns

Given the trajectory $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\}$, we can calculate the **return** G_t , of the trajectory. The naive definition of the return, called **total rewards** is the sum

$$G_t = \sum_{k=t+1}^T r_k. \quad (2.3)$$

The point is that we do not know if the terminal time T is finite or not. If T is not finite, than the cumulative reward [2.3](#) diverges. For this, we introduce the **discounted return**, defined as

$$G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} r_k \quad (2.4)$$

where $\gamma \in [0, 1]$ is called **discount rate**. This works for finite trajectory too, if we let $r_k = 0$ for $k > T$.

2.3.3 Episodes

The **episode** is the finite trajectory itself, like 2.2. The episode may stop at the states called **terminal states**. The tasks with episodes are called **episodic tasks**. Unlike episodic tasks, there are **continuing tasks**, which does not ends. These two can be expressed in unified mathematical manner. For this, in the episodic task, we must consider the agent's behavior after the terminal state: to stay in that state forever, or to take action like normal states. In this note, we follow the latter, like [Zha25].

2.4 Markov Decision Process

2.4.1 Definition

The **Markov decision process** (MDP) is the general framework that describes the stochastic dynamic systems. It is defined as following:

Definition 2.4.1.1 (Markov Decision Process)

The **Markov decision process** is the 5-tuple $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$, where each elements are defined as following:

- $\mathcal{S}$ is the state space, $\mathcal{A}(s)$ is the action space associated with state s , and $\mathcal{R}(s, a)$ is a set of rewards, associated with each state-action pair (s, a) .
- $p(s', r|s, a)$ is the dynamics of the model, which is as probability of moving to state s' from s by taking action a , and obtaining reward r . Note that the state transition probability and the reward probability can be decribed with dynamics. See 2.4.1.1
- $\pi(a|s)$ is the policy, a probability of agent choosing action a in state s .
- γ is the discount rate.

with the dynamics satisfying the *Markov property*:

$$p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t) \quad (2.5)$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t) \quad (2.6)$$

where t is the current time step.

Remark 2.4.1.1 (Transition and reward probability as dynamics)

The 4-tuple dynamics can be used to describe transition probability and reward probability as following:

$$p(s'|s, a) = \sum_{r \in \mathcal{R}(s, a)} p(s', r|s, a) \quad (2.7)$$

$$p(r|s, a) = \sum_{s' \in \mathcal{S}} p(s', r|s, a) \quad (2.8)$$

The dynamics can be either stationary or non-stationary. If the dynamics changes over time, it is non-stationary; else, it is called stationary. We'll use following notations for expectation of rewards:

- $r(s, a) = \sum_{r \in \mathcal{R}(s, a)} p(r|s, a)r$

Chapter 3

Bellman Operator

3.1 Introduction

This section introduces the Bellman equation and the Bellman optimality equation, using the Bellman operator as a main framework. This part was influenced from [LS96]. Note that we always assume that the state space and action space is finite. Although there are algorithms that can be applied to continuous states or actions (9), for simplicity, we assume finiteness. (and actually, no convergence is guaranteed for DQN.)

3.2 The State Value Function and Action Value Function

Before diving into the Bellman equation, we introduce an important concept called the **value of a state** and the **value of an action**.

Definition 3.2.0.1 (State Value)

Let the agent follows the policy π . The **state value** $v_\pi(s)$ of state s is defined as the expected return:

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s] \quad (3.1)$$

where G_t is the discounted return $G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} R_k$. The function v_π itself is called **state-value function**.

State values are used to evaluate the policy π . Policies that generate greater state values are better. Then, how can we obtain the state values? The answer is to solve the Bellman equation.

Definition 3.2.0.2 (Action Value)

Let π be a given policy. Then, the **action value** of a state-action pair is defined as

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]. \quad (3.2)$$

Remark 3.2.0.1 (Properties of Action Value)

Action value $q_\pi(s, a)$ has following properties.

- $v_\pi = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s, a)$
- $q_\pi(s, a) = \sum_{r \in \mathcal{R}(s, a)} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')$

Proof. • Use the law of total expectation ??.

- Use the result above and 3.4.0.1.

□

Actually, the 3.2.0.1 is the **Bellman equation** of state values.

3.3 Generalized Bellman Equation

Definition 3.3.0.1 (Generalized Bellman Equation)

Let $v(s)$ be a state value function. Then, the **generalized Bellman equation** is defined as following:

$$v(s) = \bigotimes_a \left(r(s, a) + \gamma \bigoplus_{s'} v(s') \right) \quad (3.3)$$

where $\bigotimes$ defines how an optimal agent should choose actions, and $\bigoplus$ defines how the value of the next state should be used in assigning value to the current state.

From here, we can define the **generalized Bellman operator** $T : \mathbb{R}^{\mathcal{S}} \rightarrow \mathbb{R}^{\mathcal{S}}$ as following:

$$[Tv](s) = \bigotimes_a \left(r(s, a) + \gamma \bigoplus_{s'} v(s') \right) \quad (3.4)$$

The **myopic policy** π with respect to a value function v is any (stochastic) policy such that $T^\pi v = Tv$ where

$$[T^\pi v](s) = \sum_a \pi(s|a) \left(r(s, a) + \gamma \bigoplus_{s'} v(s') \right) \quad (3.5)$$

For a myopic policy to exist, we require that the operator $\bigotimes$ satisfy the following property: for all functions f and states s , $\min_a f(s, a) \leq \bigotimes_a f(s, a) \leq \max_a f(s, a)$ ([LS96]). And we will only use the operator $\left[\bigoplus_{s'} v \right]^a(s) = \sum_{s' \in \mathcal{S}} p(s'|s, a) v(s')$, since we only cope with MDPs.

3.4 Bellman Equation

The value function with respect to a policy π satisfies the following Bellman equation. The Bellman equation is simply a set of linear equations that describe the relationships of each state values. Bellman equation can be obtained by using

the recursive property of return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \quad (3.6)$$

$$= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4}) + \dots \quad (3.7)$$

$$= R_{t+1} + \gamma G_{t+1}. \quad (3.8)$$

Substituting equation 3.1 with the recursive relationship, we obtain

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s] \quad (3.9)$$

$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \quad (3.10)$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s] \quad (3.11)$$

Using the law of total expectation ??, we get

$$\mathbb{E}[R_{t+1} | S_t = s] = \sum_{a \in \mathcal{A}(s)} \pi(a|s) \mathbb{E}[R_{t+1} | S_t = s, A_t = a] \quad (3.12)$$

$$= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{r \in \mathcal{R}(s,a)} p(r|s,a) r \quad (3.13)$$

and

$$\mathbb{E}[G_{t+1} | S_t = s] = \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} | S_{t+1} = s', S_t = s] p(s'|s) \quad (3.14)$$

$$= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} | S_{t+1} = s'] p(s'|s) \quad (\text{Markov property}) \quad (3.15)$$

$$= \sum_{s' \in \mathcal{S}} v_\pi(s') p(s'|s) \quad (3.16)$$

$$= \sum_{s' \in \mathcal{S}} v_\pi(s') \sum_{a \in \mathcal{A}(s)} p(s'|s,a). \quad (3.17)$$

The final result is given in the following box:

Theorem 3.4.0.1 (Bellman Equation for State-Value Function)

Let π be a policy, and let v_π be a state-value function. Then, the following Bellman equation is satisfied:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) \left(r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) v_\pi(s') \right) \quad (3.18)$$

And one can see the operator $\left[\bigotimes_{s'} v \right]^a(s) = \sum_{s' \in \mathcal{S}} p(s'|s,a) v(s')$.

3.4.1 Matrix-Vector form of Bellman Equation

We can reformulate Bellman equation in terms of matrix-vector form. For this, we rewrite the Bellman equation given in Theorem 3.4.0.1 as

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s' \in \mathcal{S}} p_\pi(s'|s) v_\pi(s'), \quad (3.19)$$

where

$$r_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{r \in \mathcal{R}(s,a)} p(r|s,a)r \quad (3.20)$$

$$p_\pi(s'|s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s)p(s'|s,a). \quad (3.21)$$

The r_π and p_π denotes the mean of immediate rewards and probability of transitioning from s to s' under policy π , respectively. Suppose that we indexed all states as s_i with $i = 1, \dots, n$, where $n = |\mathcal{S}|$. Let

$$v_\pi = \begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}, r_\pi = \begin{bmatrix} r_\pi(s_1) \\ \vdots \\ r_\pi(s_n) \end{bmatrix}, P_\pi = \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \dots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \dots & p_\pi(s_n|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \dots & p_\pi(s_n|s_n) \end{bmatrix}. \quad (3.22)$$

Then, Theorem 3.4.0.1 can be written in the following form:

Theorem 3.4.1.1 (Matrix-Vector form of Bellman Equation)

Let v_π , r_π and P_π be given as 3.22. Then, following is satisfied:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \quad (3.23)$$

Remark 3.4.1.1 (Properties of Matrix P_π)

Note that the matrix P_π given in equation 3.22 satisfies following properties:

- All the elements are nonnegative;
- The sum of the values of every row equals 1. Matrix with this property is called a *stochastic matrix*.

3.5 Bellman Optimality Equation

3.5.1 Optimal Policy

The reinforcement learning algorithms are all designed to find an optimal policy for given enviroment. Then, what is an 'optimal' policy? The definition is given below.

Definition 3.5.1.1 (Optimal Policy and Optimal State Value)

A policy π^* is optimal if and only if $v_{\pi^*}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$ and for any other policy π . The state values of π^* are the optimal state values.

Definition 3.5.1.2 (Better Policy)

A policy π_1 is **better** than a policy π_2 if and only if $v_{\pi_1}(s) \geq v_{\pi_2}(s)$ for all $s \in \mathcal{S}$.

The questions that naturally arise are:

- Existence: Does the optimal policy exists?
- Uniqueness: Is the optimal policy unique?
- Stochasticity: Is the optimal policy stochastic or deterministic?
- Algorithm: How do we obtain the optimal policy and the optimal state values?

3.5.2 Bellman Optimality Equation

The optimal policy and optimal state values can be analysed via the **Bellman optimality equation** (BOE). The optimal state values satisfies following Bellman optimality equation. This claim in fact has two logically independent statements:

1. The BOE, viewed purely as a fixed-point equation, has a unique solution (Section 3.6.2);
2. The unique solution of BOE concides with the optimal stat-value function v^* , and the derived greedy policy is optimal (Section 3.7)

Note that the Section 3.6.2 does not assume that the state-value function is optimal.

Definition 3.5.2.1 (Bellman Optimality Equation)

Let $\gamma \in (0, 1)$ be a discount rate. Then, the Bellman Optimality Equation is defined as

$$v(s) = \max_{a \in \mathcal{A}(s)} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v(s') \right) \quad (3.24)$$

$$= \max_{a \in \mathcal{A}(s)} q(s, a) \quad (3.25)$$

where $v(s), v(s')$ are unkown variables to be solved.

The Bellman optimality operator $T^* : \mathbb{R}^{\mathcal{S}} \rightarrow \mathbb{R}^{\mathcal{S}}$, uses $\otimes_a = \max_a$ as its action selection strategy. Specifically, the Bellman optimality operator is defined as following:

$$[T^*v](s) = \max_{a \in \mathcal{A}(s)} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v(s') \right) \quad (3.26)$$

3.6 Solution of The Bellman Equations

3.6.1 Contraction Mapping Theorem

We propose a **contraction mapping theorem**, the main tool for solving the Bellman equations. The contraction mapping theorem is a powerful tool for analyzing general nonlinear equations.

Definition 3.6.1.1 (Contraction Mapping)

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$. Then, the function f is a **contraction mapping** if there exists $\gamma \in (0, 1)$ such that

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\| \quad (3.27)$$

for all $x_1, x_2 \in \mathbb{R}^d$.

Now we present the contraction mapping theorem.

Theorem 3.6.1.1 (Contraction Mapping Theorem)

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a contraction mapping. Then, the following properties hold for equation $x = f(x)$:

- Existence: There exists a fixed point x^* satisfying $f(x^*) = x^*$;
- Uniqueness: The fixed point x^* is unique;
- Algorithm: The following sequence $\{x_k\}$ converges to the fixed point x^* exponentially, for any initial x_0 .

$$x_{k+1} = f(x_k) \quad (3.28)$$

Proof. Part 1. (Convergence) The sequence $\{x_k\}$ defined as $x_{k+1} = f(x_k)$ converges.

We show that the sequence $\{x_k\}$ is a Cauchy sequence; i.e., there always exists an $N \in \mathcal{N}$ such that $\|x_m - x_n\| < \epsilon$ for any $\epsilon > 0$. The Cauchy sequence has a nice property; the Cauchy sequence always converges. This can be proved by using the properties of $\mathbb{R}$, that the $\mathbb{R}$ is a complete set. We won't show this here. The following holds:

$$\|x_{k+1} - x_k\| = \|f(x_k) - f(x_{k-1})\| \quad (3.29)$$

$$\leq \gamma \|x_k - x_{k-1}\| \quad (3.30)$$

$$\leq \gamma^2 \|x_{k-1} - x_{k-2}\| \quad (3.31)$$

$$\leq \gamma^3 \|x_{k-2} - x_{k-3}\| \quad (3.32)$$

$$\vdots \quad (3.33)$$

$$\leq \gamma^k \|x_1 - x_0\| \quad (3.34)$$

Using this, we can show that

$$\|x_m - x_n\| \leq \|x_m - x_{m-1}\| + \|x_{m-1} - x_{m-2}\| + \cdots + \|x_{n+1} - x_n\| \quad (3.35)$$

$$\leq \gamma^{m-1} \|x_1 - x_0\| + \gamma^{m-2} \|x_1 - x_0\| + \cdots + \gamma^n \|x_1 - x_0\| \quad (3.36)$$

$$= \|x_1 - x_0\| (\gamma^{m-1} + \gamma^{m-2} + \cdots + \gamma^n) \quad (3.37)$$

$$= \|x_1 - x_0\| \frac{\gamma^n (1 - \gamma^{m-n})}{1 - \gamma} \quad (3.38)$$

$$\leq \|x_1 - x_0\| \frac{\gamma^n}{1 - \gamma} \quad (3.39)$$

where $m \geq n$, without loss of generality. Since $\gamma \in (0, 1)$, the last line converges to 0. That is, the sequence $\{x_k\}$ is a Cauchy sequence.

Part 2. (Existence) $\lim_{k \rightarrow \infty} x_k$ is a solution of fixed-point equation $x = f(x)$.

Let $x^* = \lim_{k \rightarrow \infty} x_k$. Then, following holds.

$$\|x^* - f(x^*)\| \leq \|x^* - x_k\| + \|f(x_k) - f(x^*)\| + \|x_k - f(x_k)\| \quad (3.40)$$

$$\leq \|x^* - x_k\| + \gamma \|x_k - x^*\| + \|x_k - f(x_k)\| \quad (3.41)$$

Using the results of *Part 1* and squeeze theorem, we can show that $x^* = f(x^*)$.

Part 3. (Uniqueness) The fixed-point x^* is unique.

Suppose there is another fixed point x' . Then, $\|x^* - x'\| = \|f(x^*) - f(x')\| \leq \gamma \|x^* - x'\|$. Since $\gamma \in (0, 1)$, the only possibility is $x^* = x'$.

□

3.6.2 Existence and Uniqueness of the Solution of Bellman Equations

Using the contraction mapping theorem, we can prove the following theorem.

Theorem 3.6.2.1 (Existence and Uniqueness of the Solution of Bellman Equations)

Let $T : \mathbb{R}^{\mathcal{S}} \rightarrow \mathbb{R}^{\mathcal{S}}$ be the generalized Bellman operator: $[Tv](s) = \bigotimes_a (r(s, a) + \gamma \bigoplus_{s'} v(s'))$. Then, if both $\bigotimes$ and $\bigoplus$ is non-expanding, i.e., $|\bigotimes_a f_1(s, a) - \bigotimes_a f_2(s, a)| \leq \max_a |f_1(s, a) - f_2(s, a)|$, and $0 \leq \gamma < 1$, the associated Bellman equation

$$v(s) = \bigotimes_a \left(r(s, a) + \gamma \bigoplus_{s'} v(s') \right) \quad (3.42)$$

always has solution and the solution is unique.

Proof. We show that the operator T is a contraction operator, and use Theorem 3.6.1.1. Let $h_v(s, a) = r(s, a) + \gamma \bigoplus_{s'} v(s')$. Then,

$$h_{v_1}(s, a) - h_{v_2}(s, a) = \gamma \bigoplus_{s'} (v_1(s') - v_2(s')) \quad (3.43)$$

and using the non-expansion assumption of $\bigoplus$,

$$|h_{v_1}(s, a) - h_{v_2}(s, a)| \leq \gamma \max_{s'} |v_1(s') - v_2(s')| = \gamma \|v_1 - v_2\|_{\infty} \quad (3.44)$$

for all (s, a) . Now, using this and non-expansion assumption of $\bigotimes$, we can get

$$|[Tv_1](s) - [Tv_2](s)| = \left| \bigotimes_a h_{v_1}(s, a) - \bigotimes_a h_{v_2}(s, a) \right| \leq \max_a |h_{v_1}(s, a) - h_{v_2}(s, a)| \leq \gamma \|v_1 - v_2\|_{\infty} \quad (3.45)$$

for all s . Therefore,

$$\|[Tv_1](s) - [Tv_2](s)\|_{\infty} \leq \gamma \|v_1 - v_2\|_{\infty} \quad (3.46)$$

□

Now, if we prove that $\max_a, \sum_{s'} \pi(s, a)$ and $\sum_{s' \in \mathcal{S}} p(s'|s, a)v(s')$ is a non-expansion operator, the existence and uniqueness of the solution of Bellman equation and Bellman optimality equation is guaranteed, respectively. TODO: Show it!

3.7 Optimal Policy, Optimal State Value and Bellman Optimality Equation

Now, we should prove that the solution of the Bellman optimality equation is really an optimal state-value function, and prove that the greedy policy derived from it is the optimal policy.

Theorem 3.7.0.1 (Optimality of v^* and π^*)

Let v^* be the solution of BOE $v = T^*v$, and let $\pi^*(a|s) = 1 \Leftrightarrow a \in \arg \max_{a'} q(s, a')$. Then, v^* is an optimal state value function, and π^* is an optimal policy. That is, for any policy π , following holds;

$$v^* = v_{\pi^*} \geq v_{\pi} \quad (3.47)$$

where v_{π} is the state value function of π , and $\geq$ is applied elementwise.

Proof. Part 1. We prove two lemmas.

First, $v_1 \leq v_2 \Rightarrow Tv_1 \leq Tv_2$ for $T = T_V^*$ or T_V^{π} elementwise. $q_1(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a)v_1(s') \leq r(s, a) + \gamma \sum_{s'} p(s'|s, a)v_2(s') = q_2(s, a)$ for all (s, a) since $p(s'|s, a) \geq 0$ and $\gamma > 0$. Then, $q_1(s, a) \leq q_2(s, a) \leq \max_a q_2(s, a)$ and $T^*v_1 = \max_a q_1(s, a) \leq T^*v_2 = \max_a q_2(s, a)$, $T^{\pi}v_1 \leq T^{\pi}v_2$.

Second, $\max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s)q(s, a) = \max_{a \in \mathcal{A}(s)} q(s, a)$. Where

$$\sum_{a \in \mathcal{A}(s)} \pi(a|s)q(s, a) \leq \sum_{a \in \mathcal{A}(s)} \pi(a|s) \max_{a'} q(s, a') \quad (3.48)$$

$$= \max_{a'} q(s, a') \quad (3.49)$$

and since $\mathcal{A}(s)$ is a finite set, there exists $a' \in \arg \max_a q(s, a)$. If we let $\pi(a'|s) = 1$ and 0 otherwise, we get $\sum_{a \in \mathcal{A}(s)} \pi(a|s)q(s, a) = q(s, a') = \max_a q(s, a)$. Therefore, $\max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s)q(s, a) = \max_{a \in \mathcal{A}(s)} q(s, a)$ and the deterministic policy π satisfying the equation exists, which is the policy π^* . Then, $T^{\pi^*}v^* = T^*v^* = v^*$. Since there exists a unique solution v_{π^*} to the equation $T^{\pi^*}v = v$, $v^* = v_{\pi^*}$.

Part 2. We show that $v^* \geq v_{\pi}$ for any policy π , elementwise. Using the first lemma of Part 1,

$$v^* = T^*v^* \geq T^{\pi}v^* \geq T^{\pi}T^{\pi}v^* \geq \dots \geq [T^{\pi}]^k v^* \quad (3.50)$$

for all $k \in \mathbb{N}$, elementwise. Using the Theorem 3.6.1.1, we can see that $[T^{\pi}]^k v^* \rightarrow v_{\pi}$, as $k \rightarrow \infty$, where v_{π} is the solution of the Bellman equation $T^{\pi}v = v$. Therefore, $v^* \geq v_{\pi}$, and this means that v^* is the optimal value function. $\square$

The important truth is that the optimal state value is unique but the optimal policy is not. The Theorem 3.7.0.1 only shows that there always exists at least one deterministic optimal policy. There still can be other optimal policies, too, stochastic or whatever.

3.8 Factors that Influence Optimal Policies

Now, we analyze the factors of BOE that influences optimal policies. The factors are: reward and discount rate.

3.8.1 Reward

Changing reward can lead to slightly different behavior of policies. For example, in the Figure 3.4(a) of [Zha25] (gridworld), one can see that the agent tries to move to the goal even though there are forbidden areas. The forbidden area gives the negative reward, but the cumulative reward is greater than going around the forbidden areas. This is why reward (signal) is important.

3.8.2 Discount Rate

High discount rate leads to far-sighted agent, and low discount rate leads to short-sighted agent. In the extreme, when γ , the agent only take into account the immediate reward.

3.8.3 Invariance of the Optimal Policy

Changing the reward may lead to different optimal policies, but this is not always the case. If we apply the same affine transformation to all rewards, the optimal policy remains the same.

Theorem 3.8.3.1 (Optimal Policy Invariance)

Let $v^* \in \mathbb{R}^{|S|}$ be the optimal state value satisfying $v^* = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v^*)$. If every reward $r \in \mathcal{R}$ goes through the same affine transformation $\alpha * r + \beta$, where $\alpha, \beta \in \mathbb{R}$ and $\alpha > 0$, the corresponding optimal state value v' is also an affine transformation of v^* :

$$v' = \alpha v^* + \frac{\beta}{1 - \gamma} \mathbf{1} \quad (3.51)$$

where $\gamma \in (0, 1)$ is the discount rate and $\mathbf{1} = [1, \dots, 1]^\top$. Due to this property, the optimal policy stays invariant.

Chapter 4

Stochastic Approximation Theory

4.1 Introduction

This chapter introduces measure-theoretic probability, convergence and several stochastic approximation theorems, which is a theoretic foundation for TD-Learning algorithms.

4.2 Prerequisite

The subsections ahead requires a basic knowledge about measure-theoretic probability, which are provided in [13](#). The probability theory was re-established by the measure theory, which enabled to define all the ambiguous concepts of probability theory rigorously. For comprehensive introduction about measure theory, see [\[Ax120\]](#) and for more information about measure-theoretic probability, see [\[BZ07\]](#).

4.3 Robbins-Monro Algorithm

Suppose we want to find the root of the equation

$$g(w) = 0. \tag{4.1}$$

There are many other iterative algorithms for finding the root of a given equation. Then, why do we need stochastic approximation? In real world problems, the expression of function g or its derivative is unknown, and we may even only be able to obtained the noisy observations of g . That is, with the original g and noise η , we can only observe $\tilde{g}(w, \eta) = g(w) + \eta$. The SGD method is one of the most well-known solution to problems like this. But here, we introduce a more general algorithms of stochastic approximation, called **Robbins-Monro algorithm**.

Theorem 4.3.0.1 (Robbins-Monro Algorithm)

Let g be a function and let $\tilde{g}(w, \eta) = g(w) + \eta$ where η is a random noise. Then, the sequence $\{w_k\}$ defined as $w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k)$ where $\{a_k\}$ is a sequence of positive coefficients almost surely converges to the root of equation $g(w) = 0$ under following conditions:

1. $0 < c_1 \leq \nabla_w g(w) \leq c_2$ for all w ;
2. $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$;
3. $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ and $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$;

where $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$.

Proof. Let w^* be the root of equation $g(w) = 0$. The RM algorithm is

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k) \quad (4.2)$$

$$= w_k - a_k [g(w_k) + \eta_k] \quad (4.3)$$

Then, we have

$$w_{k+1} - w^* = w_k - w^* - a_k [g(w_k) - g(w^*) + \eta_k]. \quad (4.4)$$

and $g(w_k) - g(w^*) = \nabla_w g(w'_k)(w_k - w^*)$ for some $w'_k \in [w_k, w^*]$. Let $\Delta_k = w_k - w^*$. The above equation becomes

$$\Delta_{k+1} = \Delta_k - a_k [\nabla_w g(w'_k)(w_k - w^*) + \eta_k] \quad (4.5)$$

$$= \Delta_k - a_k \nabla_w g(w'_k) \Delta_k + a_k (-\eta_k) \quad (4.6)$$

$$= [1 - a_k \nabla_w g(w'_k)] \Delta_k + a_k (-\eta_k). \quad (4.7)$$

Let $\alpha_k = a_k \nabla_w g(w'_k)$. Since $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$, and $\nabla_w g(w)$ is bounded, we know that $\sum_{k=1}^{\infty} \alpha_k = \infty$ and $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$. Thus, all the conditions in Theorem 4.4.0.1 are satisfied and hence Δ_k converges to zero almost surely. $\square$

- The first condition indicates that $g(w)$ monotonically increases. This ensures that the root of $g(w)$ exists and is unique. When it comes to the optimization problem such as $g(w) = \nabla_w J(w)$, the condition requires the function J to be a *convex* function. Additional $\nabla_w g(w) \leq c_2$ indicates that the gradient $\nabla_w g(w)$ is bounded.
- The second condition means that the sequence $\{a_k\}$ goes to 0 as $k \rightarrow \infty$, but not too fast.
- The third condition is mild. It just requires the noise to have mean 0 and to be square-integrable for all k .

Remark 4.3.0.1 (The Second Condition of RM Algorithm)

This is an intuitional investigation of the second condition of RM algorithm. Suppose that the observation $\tilde{g}(w_k, \eta_k)$ is always bounded. First, $\sum_{k=1}^{\infty} a_k^2 < \infty$ indicates that $a_k \rightarrow 0$ as $k \rightarrow \infty$. Since $w_{k+1} - w_k = -a_k \tilde{g}(w_k, \eta_k)$ and the observation is bounded, w_{k+1} and w_k approach each other as $k \rightarrow \infty$.

Second, $\sum_{k=1}^{\infty} a_k = \infty$ indicates that a_k should not converge to zero too fast. Using the equation $w_{k+1} - w_k = -a_k \tilde{g}(w_k, \eta_k)$ with the telescoping sum, we can get

$$w_1 - w_N = \sum_{k=1}^N a_k \tilde{g}(w_k, \eta_k) \quad (4.8)$$

If $\sum_{k=1}^{\infty} a_k < \infty$, then $|\sum_{k=1}^N a_k \tilde{g}(w_k, \eta_k)| \leq b$ for some finite upper bound b . Then,

$$|w_1 - w_N| = \left| \sum_{k=1}^N a_k \tilde{g}(w_k, \eta_k) \right| \leq b. \quad (4.9)$$

This shows that if $\sum_{k=1}^N a_k \tilde{g}(w_k, \eta_k) < \infty$ and $|w_1 - w^*| > b$ for the initial guess w_1 and the solution w^* , it is impossible to have $w_{\infty} = w^*$. That is, the condition $\sum_{k=1}^{\infty} a_k \tilde{g}(w_k, \eta_k) = \infty$ is necessary to ensure convergence given an arbitrary initial guess.

In practice, a_k is usually selected as a sufficiently small constant. This does not satisfy the second condition of RM algorithm, but can still converge in a certain sense, [Zha25].

4.4 Dvoretzky's Convergence Theorem

The proof of Theorem 4.3.0.1 depends on the **Dvoretzky's Theorem**, which will be introduced in this section.

Theorem 4.4.0.1 (Dvoretzky's Theorem)

Let $\{\alpha_k\}_{k=1}^{\infty}$, $\{\beta_k\}_{k=1}^{\infty}$ and $\{\eta_k\}_{k=1}^{\infty}$ be stochastic sequences such that $\alpha_k \geq 0$ and $\beta_k \geq 0$ for all k . Then, following sequence converges to zero almost surely

$$\Delta_{k+1} = (1 - \alpha_k)\Delta_k + \beta_k \eta_k \quad (4.10)$$

if following conditions are satisfied:

1. $\sum_{k=1}^{\infty} \alpha_k = \infty$, $\sum_{k=1}^{\infty} \alpha_k < \infty$ and $\sum_{k=1}^{\infty} \beta_k^2 < \infty$ uniformly almost surely;
2. $\mathbb{E}[\eta_k | \mathcal{H}_k]$ and $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq C$ almost surely;

where $\mathcal{H}_k = \{\Delta_k, \Delta_{k-1}, \dots, \eta_{k-1}, \dots, \alpha_{k-1}, \dots, \beta_{k-1}, \dots\}$.

Proof. The proof is based on the proof of [Zha25], which uses the convergence results of quasimartingales. Note that the theorem is proved only the case when α_k and β_k are determined by $\mathcal{H}_k$, for example, when α_k and β_k is a function of Δ_k or deterministic sequences.

Let $h_k = \Delta_k^2$. Then,

$$h_{k+1} - h_k = \Delta_{k+1}^2 - \Delta_k^2 \quad (4.11)$$

$$= (\Delta_{k+1} - \Delta_k)(\Delta_{k+1} + \Delta_k) \quad (4.12)$$

$$= (-\alpha_k \Delta_k + \beta_k \eta_k)[(2 - \alpha_k)\Delta_k + \beta_k \eta_k] \quad (4.13)$$

$$= -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2 \eta_k^2 + 2(1 - \alpha_k)\beta_k \eta_k \Delta_k. \quad (4.14)$$

Now, take expectations on both sides of the equation. Then,

$$\mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] = \mathbb{E}[-\alpha_k(2 - \alpha_k)\Delta_k^2 | \mathcal{H}_k] + \mathbb{E}[\beta_k^2 \eta_k^2 | \mathcal{H}_k] + \mathbb{E}[2(1 - \alpha_k)\beta_k \eta_k \Delta_k | \mathcal{H}_k]. \quad (4.15)$$

For simplicity, suppose that α_k and β_k are determined by $\mathcal{H}_k$. Then, we can take them out of expectation.

$$\mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] = -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] + 2(1 - \alpha_k)\beta_k \Delta_k \mathbb{E}[\eta_k | \mathcal{H}_k]. \quad (4.16)$$

Using the conditions of the theorem, we can get following results:

- $-\alpha_k(2 - \alpha_k)\Delta_k^2 \leq 0$ for $k \geq n$, for appropriate $n \in \mathbb{N}$.
- $\beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq \beta_k^2 C$;
- $2(1 - \alpha_k)\beta_k \Delta_k \mathbb{E}[\eta_k | \mathcal{H}_k] = 0$;

Therefore, we get

$$\mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] = -\alpha_k(2 - \alpha_k)\Delta_k^2 + \beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq \beta_k^2 C \quad (4.17)$$

and hence

$$\sum_{k=1}^{\infty} \mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k] \leq \sum_{k=1}^{\infty} \beta_k^2 C < \infty. \quad (4.18)$$

Now, based on the quasimartingale convergence theorem in (), $h_k = \Delta_k^2$ converges almost surely.

We now determine where Δ_k converges to.

$$\sum_{k=1}^{\infty} \alpha_k(2 - \alpha_k)\Delta_k^2 = \sum_{k=1}^{\infty} \beta_k^2 \mathbb{E}[\eta_k^2 | \mathcal{H}_k] - \sum_{k=1}^{\infty} \mathbb{E}[h_{k+1} - h_k | \mathcal{H}_k]. \quad (4.19)$$

Since the right-hand side is bounded, the left-hand side is also bounded, too. We have

$$\infty > \sum_{k=n}^{\infty} \alpha_k(2 - \alpha_k)\Delta_k^2 \geq \sum_{k=n}^{\infty} \alpha_k \Delta_k^2 \geq 0. \quad (4.20)$$

Therefore, $\sum_{k=1}^{\infty} \alpha_k \Delta_k^2$ is bounded. Since $\sum_{k=1}^{\infty} \alpha_k = \infty$, we must have $\Delta_k \rightarrow 0$ almost surely.

□

TODO: Add reference to quasimartingale convergence theorem.

4.4.1 Extension of Dvoretzky's Theorem

The following theorem extends Dvoretzky's Theorem to multivariate situations. TODO: Quote the JJS 1994. and Singh 2000.

4.5 Stochastic Gradient Descent

Part II

Algorithms

Chapter 5

Dynamic Programming

5.1 Introduction

This chapter discusses dynamic programming algorithms for finding optimal policies. Since these are dynamic programming algorithms, a perfect system model is required; this means that it is rarely used in practice, but these algorithms are important foundations of the model-free algorithms. Three algorithms will be presented;

- Value Iteration: The algorithms suggested by the contraction mapping theorem ??;
- Policy Iteration:
- Truncated Policy Iteration: The unified algorithm which includes value iteration and policy iteration as its special cases.

5.2 Value Iteration

5.2.1 Main Idea

The main idea of **value iteration** algorithms is the algorithms suggested in Theorem ??. In particular, the algorithm is

$$v_{k+1} = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad k = 0, 1, 2, \dots \quad (5.1)$$

Breaking down, this iterative algorithm has two steps;

1. *Policy Update*: The algorithm finds a policy π_{k+1} that solves the following optimization problem;

$$\arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k) \quad (5.2)$$

2. *Value Update*: With the policy π_{k+1} , the algorithm calculates a new state value function v_{k+1} ;

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \quad (5.3)$$

and this v_{k+1} is used in the next iteration.

5.2.2 Pseudocode

Algorithm 1: Value Iteration Algorithm

Input : $p(s', r|s, a)$ the dynamics of the system
 v initial guess

Output: v^*, π^* optimal state value and optimal policy

```

1  $\pi \leftarrow$  greedy policy derived from  $v$ 
2 repeat
3   for  $s \in \mathcal{S}$  do
4     for  $a \in \mathcal{A}(s)$  do
5        $q(s, a) \leftarrow \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s')$ 
        // policy update
6        $\pi(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_a q(s, a) \\ 0 & \text{otherwise} \end{cases}$ 
        // value update
7        $v(s) = \max_a q(s, a)$ 
8 until convergence of  $v$ 
9 return  $v, \pi$ 
```

Note that the value update of Algorithm 1 is simply substituting the original state values with maximum action values since we are using greedy policy.

5.2.3 Convergence

The convergence can be proved with Theorem 3.6.1.1 and Theorem ??.

5.3 Policy Iteration

5.3.1 Main Idea

The **policy iteration** is based on the policy improvement theorem. Like value iteration, the policy iteration consists of two steps:

1. *Policy Evaluation*: For a given policy π_k , we calculate the corresponding state values by solving following Bellman equation:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \quad (5.4)$$

2. *Policy Improvement*: Using the computed state value v_k , we obtain the improved policy as

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \quad (5.5)$$

and here is where the policy improvement theorem is applied.

The difference is that the value iteration is that instead of taking a state value and computing the policy from it, the policy iteration takes a policy and compute the value corresponding to the policy.

Also, note that policy iteration has another iterative algorithm inside it: the policy evaluation uses iterative algorithm for solving the Bellman equation $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$.

5.3.2 Pseudocode

Algorithm 2: Policy Iteration Algorithm

Input : $p(s', r|s, a)$ the dynamics of the system
 π initial guess

Output: v^*, π^* optimal state value and optimal policy

```

1 repeat
    // policy evaluation
2    $v' \leftarrow$  arbitrary initial guess
3   repeat
4     for each state  $s \in \mathcal{S}$  do
5        $v'(s) \leftarrow \sum_a \pi(a|s) \left[ \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v'(s') \right]$ 
6   until convergence of  $v'$ 
7    $v \leftarrow v'$ 
    // policy improvement
8   for each state  $s \in \mathcal{S}$  do
9     for each action  $a \in \mathcal{A}(s)$  do
10       $q(s, a) \leftarrow \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s')$ 
11       $a^* \leftarrow \arg \max_a q(s, a)$ 
12       $\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = a^* \\ 0 & \text{otherwise} \end{cases}$ 
13 until convergence of  $v$ 
14 return  $v, \pi$ 
```

5.3.3 Convergence

TODO: convergence proof of policy iteration theorem. This will be done after studying the operator version of Bellman Equation.

5.4 Truncated Policy Iteration

5.4.1 Main Idea

The **truncated policy iteration** is a more general algorithm including value iteration and policy iteration as its special cases. The algorithm has only one difference with policy iteration: in the policy evaluation step, the truncated policy iteration iterates only finite times, even when the convergence condition is not met. Truncated policy iteration algorithm is one of the instances of **general policy iteration**, which is an abstract concept of interleaving evaluation and improvement.

The perspective of looking at the truncated policy iteration as the generalization of policy iteration and value iteration can

be derived from the similiarity between value iteration and policy iteration. With the assumption that $v_{\pi}^{(0)} = v_0 = v_{\pi_0}$,

$$v_{\pi_1}^{(0)} = v_0 \quad (5.6)$$

$$\text{value iteration} \leftarrow v_1 \leftarrow v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \quad (5.7)$$

$$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \quad (5.8)$$

$$v_{\pi_1}^{(3)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(2)} \quad (5.9)$$

$$\vdots \quad (5.10)$$

$$\text{truncated policy iteration} \leftarrow \bar{v}_1 \leftarrow v_{\pi_1}^{(j+1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j)} \quad (5.11)$$

$$\vdots \quad (5.12)$$

$$\text{policy iteration} \leftarrow v_{\pi_1} \leftarrow v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)} \quad (5.13)$$

$$(5.14)$$

While value iteration computes only once and update the state value and policy iteration iterates until convergence and use it as state value of policy, the truncated policy iteration iterates only finite number of times.

5.4.2 Pseudocode

Algorithm 3: Truncated Policy Iteration Algorithm

Input : $p(s', r|s, a)$ the dynamics of the system

π initial guess

Output: v^*, π^* optimal state value and optimal policy

```

1 repeat
    // policy evaluation
2    $v' \leftarrow$  arbitrary initial guess
3   for  $k = 0, 1, \dots, k_{\text{truncate}} - 1$  do
4       for each state  $s \in \mathcal{S}$  do
5            $v^{(k+1)}(s) \leftarrow \sum_a \pi(a|s) \left[ \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v^{(k)}(s') \right]$ 
6    $v \leftarrow v^{(k_{\text{truncate}})}$ 
    // policy improvement
7   for each state  $s \in \mathcal{S}$  do
8       for each action  $a \in \mathcal{A}(s)$  do
9            $q(s, a) \leftarrow \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s')$ 
10       $a^* \leftarrow \arg \max_a q(s, a)$ 
11       $\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = a^* \\ 0 & \text{otherwise} \end{cases}$ 
12 until convergence of  $v$ 
13 return  $v, \pi$ 

```

5.4.3 Convergence

TODO: Convergence proof of truncated policy iteration.

Chapter 6

Monte Carlo Methods

6.1 Introduction

This chapter introduces a **Monte Carlo methods** for reinforcement learning. These algorithms all starts from trying to obtain the optimal policy by solving the Bellman optimality equation. The difference of pure dynamic programming algorithms and Monte Carlo methods is that the latter do not require a perfect dynamics of the model, i.e., it is *model-free*.

Then, how do we get the optimal policy without a perfect model? We use the data that the model gives us. Using the data, we can convert the policy iteration algorithm 2 to the model-free version. Specifically, we estimate the action value $q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$ using Monte Carlo sampling.

6.2 Monte Carlo Basic

6.2.1 Main Idea

The **Monte Carlo basic** is the simplest Monte Carlo method algorithm. Suppose that we run the agent starting from state-action pair (s, a) , following policy π , and let $g_\pi^{(i)}(s, a)$ be the return of the episode. Then, we can approximate the action value $q_\pi(s, a)$ as following:

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_\pi^{(i)}(s, a). \quad (6.1)$$

If the number n is sufficiently large, the approximation will be sufficiently accurate. This can be guaranteed due to the law of large numbers.

6.2.2 Pseudocode

Algorithm 4: Monte Carlo Basic

Input : π_0 initial guess
Output: π^* optimal policy

```
1 repeat
2   foreach  $s \in \mathcal{S}$  do
3     foreach  $a \in \mathcal{A}(s)$  do
4       // policy evaluation
5       collect sufficiently many episodes starting from  $(s, a)$ , following policy  $\pi_k$ 
6        $q_{\pi_k}(s, a) \leftarrow$  the average return of all episodes starting from  $(s, a)$ 
7     // policy improvement
8      $\pi_{k+1}(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} q_{\pi_k}(s, a') \\ 0 & \text{otherwise} \end{cases}$ 
9   until convergence of  $\pi$ 
10  return  $\pi$ 
```

6.2.3 Convergence

6.2.4 Pros and Cons

Cons:

- Very inefficient sample usage. Due to this property, MC-Basic is not used in practical. To deal with this, we use MC with exploring start, which is presented in next section.

6.3 Monte Carlo with Exploring Starts

6.3.1 Main Idea

The MC-Basic algorithms is not sample-efficient, since it only updates only q value of one state-action pair each episode. To deal with this low efficiency, we use all the *visit* to the state-action pair in each episode.

Suppose that the agent has gone through following trajectory:

$$s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} s_4 \xrightarrow{a_4} \dots \quad (6.2)$$

To increase the sample efficiency, we use the visits to the state-action pairs in the trajectory. That is, using the above episode, we not only use (s_1, a_1) for estimating $q(s_1, a_1)$ but use (s_2, a_2) for estimating $q(s_2, a_2)$, and so on. There are two different ways to do this:

- The first way is to only use the state-action pairs when it is first encountered. This is called *first-visit method*;
- The second way is to use all the visit to the state-action pairs. This is called *every-visit method*.

Also, we update the estimate of q values after each episode ends. This falls into the scope of *generalized policy iteration*, since we are using the value approximation even it is not so accurate.

6.3.2 Pseudocode

Algorithm 5: Monte Carlo with Exploring Starts (every-visit)

Input : π_0 initial policy
Output: π^* optimal policy

```

// initialize
1 foreach valid state-action pair  $(s, a)$  do
2    $\pi \leftarrow \pi_0$ 
3    $q(s, a) \leftarrow$  initial value
   // holds the sum of returns of (sub)episode starting from  $(s, a)$ 
4    $\text{Ret}(s, a) \leftarrow 0$ 
   // counts the number of visits of  $(s, a)$ 
5    $N(s, a) \leftarrow 0$ 
6 repeat
7   select a starting valid state-action pair  $(s, a)$ , ensuring that all the pairs can be possibly selected
8   generate an episode starting from  $(s, a)$ , following current policy  $\pi$ :  $s_0, a_0, r_1, s_1, a_1, \dots, s_{T-1}, a_{T-1}, r_T$ 
9    $G \leftarrow 0$ 
10  for  $t = T - 1, T - 2, \dots, 0$  do
11     $G \leftarrow r_{t+1} + \gamma G$ 
12     $\text{Ret}(s_t, a_t) \leftarrow \text{Ret}(s_t, a_t) + G$ 
13     $N(s_t, a_t) \leftarrow N(s_t, a_t) + 1$ 
    // policy evaluation
14     $q(s_t, a_t) \leftarrow \frac{\text{Ret}(s_t, a_t)}{N(s_t, a_t)}$ 
    // policy improvement
15     $\pi_{k+1}(a_t|s_t) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} q_{\pi_k}(s, a') \\ 0 & \text{otherwise} \end{cases}$ 
16 until convergence

```

6.3.3 Implementation Details

The exploring-start condition can be achieved by random start state.

6.3.4 Pros and Cons

Pros:

- Better sample efficiency than MC-Basic;
- All state-action pairs can be visited due to the exploring-start, and this makes the estimation more accurate.

Cons:

- Only applicable to episodic tasks, which is a common drawback for all MC methods. This property leads to following problem, too;
- High variance since the return holds all the noises and randomnesses, and the return is used for update;
- If the exploring start condition cannot be satisfied, the MC with Exploring Starts cannot be used. This is common in real-world problems. To deal with this, another algorithms without exploring starts will be introduced next section;

6.4 Monte Carlo without Exploring Starts

6.4.1 Main Idea

To deal with the exploring starts, we use ε -greedy function for exploring. That is, instead of using the greedy policy for training, we train the ε -greedy policy. This little probability of choosing non-greedy action is required, since the greedy action we have depends only on current informations. That is, there can be a better choice of action selection, since the action values are not perfectly known. Actually, balancing the choice of greedy and non-greedy action is a very important problem in reinforcement learning. This problem is called the problem of **exploitation** and **exploration**.

Remark 6.4.1.1 (Problem of Exploitation and Exploration)

To **exploit** is to use the current information and select the greedy action. To **explore** is to select the action other than greedy action. Since the true state values cannot be known in training, the action that looks greedy may not lead to the highest total reward. If the agent exploits too much, the agent may find only suboptimal policies. If the agent explores too much, the agent won't be able to find the optimal policy. As aforementioned, the balance between exploitation and exploration is critical.

6.4.2 Pseudocode

Algorithm 6: Monte Carlo with ε -greedy policy

Input : π_0 initial policy
 $\varepsilon \in (0, 1)$ probability of exploration
Output: π^* optimal policy

```

// initialize
1 foreach valid state-action pair  $(s, a)$  do
2    $\pi \leftarrow \pi_0$ 
3    $q(s, a) \leftarrow$  initial value
   // holds the sum of returns of (sub)episode starting from  $(s, a)$ 
4    $\text{Ret}(s, a) \leftarrow 0$ 
   // counts the number of visits of  $(s, a)$ 
5    $N(s, a) \leftarrow 0$ 
6 repeat
7   select a starting valid state-action pair  $(s, a)$ 
8   generate an episode starting from  $(s, a)$ , following current policy  $\pi$ :  $s_0, a_0, r_1, s_1, a_1, \dots, s_{T-1}, a_{T-1}, r_T$ 
9    $G \leftarrow 0$ 
10  for  $t = T - 1, T - 2, \dots, 0$  do
11     $G \leftarrow r_{t+1} + \gamma G$ 
12     $\text{Ret}(s_t, a_t) \leftarrow \text{Ret}(s_t, a_t) + G$ 
13     $N(s_t, a_t) \leftarrow N(s_t, a_t) + 1$ 
    // policy evaluation
14     $q(s_t, a_t) \leftarrow \frac{\text{Ret}(s_t, a_t)}{N(s_t, a_t)}$ 
    // policy improvement
15     $\pi_{k+1}(a_t|s_t) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A(s)|} & \text{if } a = \arg \max_{a'} q(s, a) \\ \frac{\varepsilon}{|A(s)|} & \text{otherwise} \end{cases}$ 
16 until convergence

```

6.4.3 Implementation Details

The problem with ε -greedy policy is that it even if the trained policy is optimal in the spaces of ε -greedy policies, the policy still has a probability of exploration. To deal with this, ε -greedy policy with GLIE (Greedy in the Limit with Infinite Exploration).

Definition 6.4.3.1 (GLIE Condition)

When following two conditions are satisfied, the policy is called GLIE.

- Infinite Exploration: Every state-action pair must be visited and explored infinitely many times as the training steps go to infinity (i.e., $t \rightarrow \infty$).
- Limit Greediness: As the learning agent experiences more of the environment, the policy must gradually shift from random exploration to exploiting what it knows, eventually converging entirely to a greedy (optimal) policy.

The most simple way to establish this condition is to use ε -greedy policy with decreasing ε . We can decay the ε linearly, exponentially or whatever. TODO: search for several GLIE policies.

According to the approximation theories in Chapter 4, we can also estimate the state values using the stochastic approximation, without using the $\text{Ret}(s, a)$ and $N(s, a)$ in Algorithm 6. The mainly used approximation is to update the action value as

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(G_t - q(s_t, a_t)) \quad (6.3)$$

where α is a fixed constant.

6.4.4 Pros and Cons

Pros:

- Better sample efficiency than MC-Basic;
- Does not require exploring starts, and still explores sufficiently;

Cons:

- Only applicable to episodic tasks, which is a common drawback for all MC methods. This property leads to following problem, too;
- High variance since the return holds all the noises and randomnesses, and that return is used for update;

Chapter 7

Temporal Difference Methods

7.1 Introduction

This chapter presents the temporal-difference (TD) methods for reinforcement learning. The main difference of MC and TD is that TD learning is incremental, while MC is not. That is, TD methods are applicable on continuous tasks, unlike MC method.

7.2 TD Learning of State Values

The general algorithm of TD method with state value is given in this section. In practice, we use TD method for action values, but the one given here is a foundation of them.

7.2.1 Main Idea

Given a policy π , we estimate the state value with following update equation, along the trajectory $(s_0, r_1, s_1, r_2, \dots, r_t, s_t, \dots)$.

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \left[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) \right] \quad (7.1)$$

$$v_{t+1}(s) = v_t(s), \text{ for all } s \neq s_t \quad (7.2)$$

This equation can be viewed as a certain stochastic approximation of the solution of the Bellman equation.

Remark 7.2.1.1 (Derivation of TD-Learning Algorithm)

Recall the definition of Bellman equation of state values (Definition 3.1). Modifying this equation a bit, we can write this equation as following:

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s], \quad s \in \mathcal{S} \quad (7.3)$$

Now define the function

$$g(v_\pi(s_t)) = v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t] \quad (7.4)$$

and then our goal is to solve $g(v_\pi(s_t)) = 0$ to obtain the state value $v_\pi(s_t)$.

Since we can get r_t and s_{t+1} in the trajectory, we can get the noisy observation $\tilde{g}(v_\pi(s_t))$, at the following equation shows the separation of true term and the noise term into linear combination.

$$\tilde{g}(v_\pi(s_t)) = v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})] \quad (7.5)$$

$$= (v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t]) \quad (7.6)$$

$$+ (\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t] - [r_{t+1} + \gamma v_\pi(s_{t+1})]) \quad (7.7)$$

$$= g(v_\pi(s_t)) + \eta(s_t) \quad (7.8)$$

Therefore the RM algorithm 4.3.0.1 can be applied:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \tilde{g}(v_t(s_t)) \quad (7.9)$$

$$= v_t(s_t) - \alpha_t(s_t) (v_t(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]) \quad (7.10)$$

Now the only difference between equation 7.1 and equation 7.10 is the bootstrap: equation 7.1 bootstraps (uses $v_t(s_{t+1})$), but the equation 7.10 does not (uses $v_\pi(s_{t+1})$). The equation with bootstrap term still guarantees the almost sure convergence to the solution of $g(v_\pi(s_t)) = 0$. This is proved in Theorem 7.2.1.1.

Theorem 7.2.1.1 (Convergence of TD Algorithm)

Given a policy π , by the TD algorithm in equation 7.1, $v_t(s)$ converges almost surely to $v_\pi(s)$ as $t \rightarrow \infty$ for all $s \in \mathcal{S}$ if $\sum_t \alpha_t(s) = \infty$ and $\sum_t \alpha_t^2(s) < \infty$ for all $s \in \mathcal{S}$.

Note that the condition $\sum_t \alpha_t(s) = \infty$ requires all states to be visited infinitely many times, since $\alpha_t(s) > 0$ if s is visited in time t and $\alpha_t(s) = 0$ if not.

Proof. We use the extension Dvoretzky's Convergence Theorem ???. To do so, we first organize the stochastic processes. Let $\Delta_t(s) = v_t(s) - v_\pi(s)$. Then,

$$\Delta_{t+1}(s_t) = (1 - \alpha_t(s_t))\Delta_t(s_t) + \alpha_t(s_t)(r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s_t)) \quad (7.11)$$

$$= (1 - \alpha_t(s_t))\Delta_t(s_t) + \alpha_t(s_t)\eta_t(s_t) \quad (7.12)$$

$$\Delta_{t+1}(s) = \Delta_t(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s)\eta_t(s), \quad s \neq s_t, \text{ where } \alpha_t(s) = 0 \text{ and } \eta_t(s) = 0 \quad (7.13)$$

Therefore, regardless of whether $s = s_t$, we get unified equation $\Delta_{t+1}(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s)\eta_t(s)$. To prove this stochastic process converges almost surely to 0 with Theorem ???, we must show that this process satisfies the conditions required.

1. Assumption of this theorem satisfies the first condition.
2. Due to the Markov property, the noise term η_T does not depend on the historical information $\mathcal{H}_t$. Therefore, $\mathbb{E}[\eta_t(s) | \mathcal{H}_t] = \mathbb{E}[\eta_t(s)]$. For $s \neq s_t$, we have $\eta_t(s) = 0 \leq \gamma \|\Delta_t(s)\|_\infty$. For $s = s_t$, we have

$$\mathbb{E}[\eta_t(s)] = \mathbb{E}[\eta_t(s_t)] \quad (7.14)$$

$$= \mathbb{E}[r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s_t) | S_t = s_t] \quad (7.15)$$

$$= \mathbb{E}[r_{t+1} + \gamma v_t(s_{t+1}) | S_t = s_t] - v_\pi(s_t) = \gamma \mathbb{E}[v_t(s_{t+1}) - v_\pi(s_{t+1}) | S_t = s_t] \quad (7.16)$$

$$= \gamma \sum_{s' \in \mathcal{S}} p(s' | s_t) [v_t(s') - v_\pi(s')] \quad (7.17)$$

Applying $|\cdot|$ to both sides of equation, we get

$$|\mathbb{E}[\eta_t(s_t)]| = \gamma \left| \sum_{s' \in \mathcal{S}} p(s'|s_t)[v_t(s') - v_\pi(s')] \right| \quad (7.18)$$

$$\leq \gamma \sum_{s' \in \mathcal{S}} p(s'|s_t) \max_{s' \in \mathcal{S}} |v_t(s') - v_\pi(s')| \quad (7.19)$$

$$= \gamma \max_{s' \in \mathcal{S}} |v_t(s') - v_\pi(s')| \quad (7.20)$$

$$= \gamma \|v_t(s') - v_\pi(s')\|_\infty \quad (7.21)$$

$$= \gamma \|\Delta_t(s)\|_\infty \quad (7.22)$$

3. TODO

□

Note that this algorithm can only estimate the state value of current, given policy. This can be seen as the policy evaluation step of policy iteration method (Section 5.3). To find the optimal state values, we can add the policy improvement step here.

7.3 Sarsa

Now, we introduce the most basic TD-algorithm, the **Sarsa**. Its name originates from the tuple (s, a, r, s', a') it uses.

In the last section, the TD algorithm of learning state value was mainly discussed. Here, we directly learn the action values instead of state values, since it can be combined with a policy improvement step to learn optimal policies.

7.3.1 Main Idea

Suppose we have trajectory generated from policy π : $(s_0, a_0, r_1, s_1, a_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots)$. Note that the trajectory does not have to end. We use following TD-update every step for estimating the action values:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t) [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) - q_t(s_t, a_t)] \quad (7.23)$$

$$q_{t+1}(s, a) = q_t(s, a), \text{ for all } (s, a) \neq (s_t, a_t) \quad (7.24)$$

In fact, the Sarsa solves following Bellman equation in a stochastic approximation manner.

$$q_\pi(s, a) = \mathbb{E}[R + \gamma q_\pi(S', A') | s, a] \text{ for all } (s, a) \quad (7.25)$$

Equation 7.25 is the Bellman equation about action values. The proof is given below.

Proof. Note that

$$p(s', a' | s, a) = p(s' | s, a) p(a' | s', s, a) \quad (7.26)$$

$$= p(s' | s, a) p(a' | s') \text{ (due to the Markov property)} \quad (7.27)$$

$$= p(s' | s, a) \pi(a' | s'). \quad (7.28)$$

Using this, the equation of Theorem ?? can be written as

$$q_\pi(s, a) = \sum_r rp(r|s, a) + \gamma \sum_{s', a'} q_\pi(s', a')p(s', a'|s, a) \quad (7.29)$$

□

7.3.2 Pseudocode

Algorithm 7: Sarsa

Input : $\alpha_t(s, a) \in (0, 1]$ step size parameter satisfying RM condition

Output: q_* the optimal policy

```
// initialize
1 foreach  $(s, a) \in \mathcal{S} \times \mathcal{A}$  do
2    $q(s, a) \leftarrow \begin{cases} \text{arbitrary,} & \text{if } s \in \mathcal{S}^+ \text{ and } a \in \mathcal{A}(s) \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$ 
3  $\pi \leftarrow$  policy derived from  $q$  (e.g.  $\epsilon$ -greedy, softmax, GLIE)
4 foreach episodes do
5   initialize  $s$ 
6   choose action  $a$  from  $s$  using policy  $\pi$ 
7   repeat
8     take action  $a$ , observe  $r, s'$ 
9     choose action  $a'$  from  $s'$  using policy derived from  $\pi$ 
10    // update q value
11     $q(s, a) \leftarrow q(s, a) + \alpha_t(s, a)[r + \gamma q(s', a') - q(s, a)]$ 
12    // update policy
13    update  $\pi$  according to  $q(s, a)$   $s \leftarrow s'$ 
14     $a \leftarrow a'$ 
15  until  $s$  is in terminal state
16 return  $q \simeq q_*$ 
```

7.3.3 Implementation Detail

I usually initialize q value without minding if the state is terminal or not. While training the agent, when the agent meets terminal state, I just check it in place and set the $q_t(s_{t+1}, a_{t+1}) = 0$. Actually I don't even change the q value of terminal state to 0. I just leave it untouched.

7.3.4 Convergence

Theorem 7.3.4.1 (Convergence of Sarsa)

Given a policy π , by the Sarsa algorithm 7.23, $q_t(s, a)$ converges almost surely to the action value $q_\pi(s, a)$ as $t \rightarrow \infty$ for all (s, a) if $\sum_t \alpha_t(s, a) = \infty$ and $\sum_t \alpha_t^2(s, a) < \infty$ for all (s, a) .

Proof. TODO: organize the proof!

□

7.3.5 Pros and Cons

Pros:

- It learns safer paths by accounting for penalties from exploratory moves. This can be checked with cliff walking of [Sut20], with comparison to Q-Learning.

Cons:

- It learns suboptimal policies first, due to the behavior of searching for safe paths in the initial training process.
- The variance can be high, since in the initial training process the q values are very noisy and inaccurate, and the agent use the q value of it directly in the update. This can be dealt with the Expected Sarsa 7.5.

7.4 Q-Learning

In this section we learn another TD-learning algorithm, the **Q-Learning**. Unlike Sarsa, Q-Learning bootstraps the q value using the maximum action value of current state.

7.4.1 Main Idea

The Q-Learning uses following update equation for action values:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t) \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1}, a)} q_t(s_{t+1}, a) - q_t(s_t, a_t) \right] \quad (7.30)$$

$$q_{t+1}(s, a) = q_t(s, a), \text{ for all } (s, a) \neq (s_t, a_t) \quad (7.31)$$

The Q-Learning solves following Bellman optimality equation:

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) | S_t = s, A_t = a \right] \quad (7.32)$$

Proof. Check for [Zha25] pg.140.

□

Remark 7.4.1.1 (On-policy and Off-policy)

As one can see, the Sarsa solves Bellman equation and then make policy update, but Q-Learning directly learns the optimal policy. The **behavior policy**, is used to make the trajectory. The **target policy** is the policy that learns the optimal policy. When the behavior policy and target policy equals, the algorithms is called **on-policy** algorithm. If not, it is called **off-policy**. Off policy can separate the exploration with target policy. The target policy only has to care about choosing the best action in current situation. Since the off-policy uses policy other than its target policy, it requires some techniques such as **importance sampling** and **tree-backup** algorithms. These will be covered later. The on-policy algorithms cannot be implemented as an **offline** fashion, since it requires real-time interaction with environment. The off-policy algorithms, in contrast, can be implemented as an offline or online fashion.

7.4.2 Pseudocode

Algorithm 8: Q-Learning

Input : $\alpha_t(s, a) \in (0, 1]$ step size parameter satisfying RM condition
 b behavior policy
 π target policy
Output: q_* the optimal policy

```
// initialize
1 foreach  $(s, a) \in \mathcal{S} \times \mathcal{A}$  do
2    $q(s, a) \leftarrow \begin{cases} \text{arbitrary,} & \text{if } s \text{ is not in terminal states} \\ 0 & \text{if } s \text{ is in terminal states} \end{cases}$ 
3  $s \leftarrow \text{initialize}$ 
4 foreach episode do
5   repeat
6     choose action  $a$  from  $s$  using behavior policy  $b$ 
7     observe state  $s'$  and reward  $r$ 
8     // update q value
9      $q(s, a) \leftarrow q(s, a) + \alpha_t(s, a) [r + \gamma \max_a q(s', a) - q(s, a)]$  // update target policy
10    update target policy according to  $q$ 
11     $s \leftarrow s'$ 
12 until  $s$  is not terminal state
```

7.4.3 Implementation Detail

The behavior policy can be any policy that explores well, but in practice we use decaying ϵ -greedy due to the convergence rate. (I guess...)

7.4.4 Convergence

TODO: Convergence of Q-Learning

7.4.5 Pros and Cons

Pros:

- It is off-policy. Whatever the behavior policy does, the target policy learns the optimal policy.
- Does not fears getting penalty, as opposed to Sarsa. This can be checked in [\[Sut20\]](#), cliff walking.

Cons:

- Can suffer from **maximization bias**, since the Q-Learning uses maximum q value for bootstrapping. That is, the agent keeps overestimate the q values, which can lead to slow convergence. For more information, see [\[Sut20\]](#). To cope with this, we can use **Double Q-Learning**.

7.5 Expected Sarsa

7.5.1 Main Idea

Expected Sarsa is very similar to Sarsa, but bootstraps the next action values with the expectation of other action values, instead of choosing only one action.

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t) \left[r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)] - q_t(s_t, a_t) \right] \quad (7.33)$$

$$q_{t+1}(s, a) = q_t(s, a), \text{ for all } (s, a) \neq (s_t, a_t) \quad (7.34)$$

This decreases the variance of Sarsa.

Chapter 8

Function Approximation Methods

8.1 Introduction

In this chapter, value function approximation methods will be covered. Unlike the tabular methods, the approximate method tries to approximate the state value or action value of state-action pairs, using a function. This solves the curse of dimensionality, which is a critical problem for tabular methods.

But there are tradeoffs, too. For example, the convergence property breaks down easily. When off-policy, function approximation, and bootstrapping meets together, (**the deadly triad**, [Sut20]), the algorithm lost its convergence property and fails to find optimal policy. There are various ways to solve this problem. Some solve this with practical methods (replay buffer, PER, double learning, etc.), and some solve this theoretically (GTD2, Emphatic-TD, etc.). I'll try to cover both of them in this note.

8.2 TD Learning of State Values based on Function Approximation

Methods of estimating the state value with a function approximation is introduced in this section.

8.2.1 Objective Function

Let $v_\pi(s)$ be the true state value function and let $\hat{v}_\pi(s, w)$ be the approximation of it with a parameter w . To find the optimal parameter w which best approximates the true state value function, we minimize the objective function. In particular, the objective function is given as

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2], \quad (8.1)$$

where S is a random variable of state space. Then what is the distribution of a random variable S ? We can use *stationary distribution* of Markov Process. Stationary distribution of a Markov Process describes the long-term behavior of a Markov decision process. That is, the probability of agent being located at certain state after a long execution can be described by this distribution.

Let $d_\pi(s)$ be the stationary distribution of the Markov process under policy π . Then, the objective function can be written

as

$$J(w) = \sum_{s \in \mathcal{S}} d_{\pi}(s) (v_{\pi}(s) - \hat{v}(s, w))^2 \quad (8.2)$$

TODO: Why stationary distribution? + Tsitsiklis and van Roy + Chapter 6

8.3 Deep Q-Learning

8.3.1 Main Idea

The **Deep Q-Learning** [Mni+13] is one of the earliest and successful deep reinforcement learning algorithms. Surprisingly, DQN algorithm are not guaranteed to converge, since the deadly triad perfectly fits into the algorithms' condition: it bootstraps, it is off-policy, and it uses function approximation. But there are practical methods for calming down this instability. But still note that the algorithm is not guaranteed to converge.

8.3.2 Objective Function

The objective function which DQN minimizes is given as following:

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right], \quad (8.3)$$

where $\hat{q}$ is the approximation function of action values and (S, A, R, S') are random variables denoting state, action, reward and next state, respectively. Note that the squared term is a Bellman optimality error.

Now, we can calculate the gradient and minimize the loss with gradient descents... but take a look. Unfortunately, the max operation is not differentiable. To resolve this, we freeze the parameter of a network inside the max operation for a short time. Specifically, we introduce two networks: *target network* and *main network*. Using the main network $\hat{q}(s, a, w)$ and $\hat{q}(s, a, w_T)$, we can re-write the objective function as following:

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right)^2 \right], \quad (8.4)$$

and the gradient can be computed about w . The parameters of target network and main network are synchronized regularly.

There is one more technique required for DQN to be trained. The *experience replay* collects the (s, a, r, s') pair, and then samples the experiences with uniform random distribution. Why is this technique required?

- [Zha25] says: Compare equation 8.4 and 8.2. Since we don't know the probability distribution of (S, A, R, S') , we just simply set it as uniform distribution. However, the state-action pairs are strongly correlated, and not uniformly distributed in single trajectory. To break the correlation, we uniformly draw samples from the replay buffer.

Also, since we can reuse the experiences, the sample efficiency naturally increases.

8.3.3 Pseudocode

Algorithm 9: Deep Q-Learning

Input : b behavior policy
 C sync rate
Output: $\hat{q}(s, a, w)$ approximated action values

```

1 store the samples generated by  $b$  in a replay buffer  $\mathcal{B} = \{(s, a, r, s')\}$ 
2 foreach iteration do
3   uniformly draw a mini batch  $B$  of samples from  $\mathcal{B}$ 
4    $L \leftarrow 0$ 
5   foreach sample  $(s, a, r, s') \in B$  do
6      $y_T \leftarrow r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$ 
7      $L \leftarrow L + \left( y_T - \hat{q}(s, a, w) \right)^2$ 
8   minimize  $L$  by updating parameter  $w$ 
9   set  $w_t = w$  every  $C$  iterations
10 return  $\hat{q}$ 

```

8.3.4 Implementation Details

Since the original DQN algorithm is highly unstable, there are several techniques to stabilize the algorithm.

- Prioritized experience replay (PER)
- Double DQN
- Dueling DQN
- Distributional DQN

8.3.5 Pros and Cons

Pros:

-

Cons:

- Catastrophic forgetting: This is a common problems in neural networks.
- No convergence guaranteed: Note that DQN satisfies the deadly triad condition. And also, the algorithm does not even use the true gradient, since the target network is freezed for a moment. This is called a **semi-gradient** method, and no convergence is guaranteed for this method. For more information see [\[Sut20\]](#).

Chapter 9

Policy-based Methods

9.1 Introduction

Policy gradient method, unlike value-based methods, tries to predict the optimal policy itself, instead of solving the Bellman equation. It has many advantages over value-based methods; It is more efficient in handling large state-action spaces, and has stronger generalization abilities.

9.2 Metrics for Defining Optimal Policies

The value-based methods uses state value and action value to define the optimal policies. New metrics for defining optimal policies are required for policy gradient methods, since they does not use the state and action values.

There are two metrics given in [Zha25].

1. average state value: $\bar{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = \mathbb{E}_{S \sim d}[v_\pi(S)]$ where $d(s)$ is the weight of state s . $d(s)$ can be independent of the policy π , denoted as $d_0(s)$ and v_0 , or dependent, for example, stationary distribution.;
2. average reward: $\bar{r}_\pi = \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}_{S \sim d_\pi}[r_\pi(S)]$ where d_π is the stationary distribution and $r_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s, \theta) r(s, a)$.

Actually the two metrics are equivalent in discounted case $\gamma < 1$. In particular, $\bar{r}_\pi = (1 - \gamma)\bar{v}_\pi$. Proofs can be found at [Zha25].

9.2.1 Average State Value

Average state value has two equivalent expressions.

- Suppose the agent has collected rewards $\{R_{t+1}\}_{t=0}^{\infty}$. Then,

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] = \sum_{s \in \mathcal{S}} d(s) \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s\right] \quad (9.1)$$

$$= \sum_{s \in \mathcal{S}} d(s) v_{\pi}(s) \quad (9.2)$$

$$= \bar{v}_{\pi} \quad (9.3)$$

- $\bar{v}_{\pi}$ can be rewritten as the inner product of two vectors, $v_{\pi} = [\dots, v_{\pi}(s), \dots]$ and $d = [\dots, d(s), \dots]$.

$$\bar{v}_{\pi} = d^{\top} v_{\pi} \quad (9.4)$$

9.2.2 Average Reward

Average reward has two equivalent expressions.

- Suppose the agent has collected rewards $\{R_{t+1}\}_{t=0}^{\infty}$. Then,

$$\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right] = \bar{r}_{\pi} \quad (9.5)$$

The proof can be found at [\[Zha25\]](#).

- Same as before, $\bar{r}_{\pi}$ can be rewritten as the inner product of two vectors.

9.3 Policy Gradient Theorem

The policy gradient method mainly depends on the **policy gradient theorem**.

Theorem 9.3.0.1 (Policy Gradient Theorem)

Let $J(\theta)$ be one of the metrics proposed in Section 9.2. Then, the gradient of $J(\theta)$ is

$$\nabla_{\theta} J(\theta) = \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}(s)} \nabla_{\theta} \pi_{\theta}(a|s) q_{\pi_{\theta}}(s, a) \quad (9.6)$$

where η is a state distribution. Moreover, the equivalent equation expressed in terms of expectation exists:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S \sim \eta, A \sim \pi_{\theta}(\cdot|S)} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) q_{\pi_{\theta}}(S, A) \right] \quad (9.7)$$

Lemma 9.3.0.1.1 (Gradient of $v_{\pi}(s)$)

In the discounted case, the following holds for any $s \in \mathcal{S}$

$$\nabla_{\theta} v_{\pi_{\theta}}(s) = \sum_{s' \in \mathcal{S}} P_{\pi_{\theta}}(s'|s) \sum_{a \in \mathcal{A}(s')} \nabla_{\theta} \pi_{\theta}(a|s') q_{\pi_{\theta}}(s', a) \quad (9.8)$$

where

$$P_{\pi_{\theta}}(s'|s) = \sum_{k=0}^{\infty} \gamma^k [P_{\pi_{\theta}}^k]_{ss'} = [(I_n - \gamma P_{\pi_{\theta}})^{-1}]_{ss'} \quad (9.9)$$

is the discounted total probability of transitioning from s to s' under policy π_{θ} . $[P_{\pi_{\theta}}^k]_{ss'}$ denotes the probability of transitioning from s to s' using exactly k steps under policy π_{θ} .

Proof. Try to naively differentiate $v_{\pi_{\theta}}(s) = \sum_{a \in \mathcal{A}(s)} \pi_{\theta}(a|s) q_{\pi}(s, a)$. Then, using the relationship between state value function and action value function, you'll be able to derive another $v_{\pi_{\theta}}$ on the right-hand side. Like the derivation of vector form of Bellman equation, use matrix-vector form to solve the linear equation. For more information, see [Zha25]. $\square$

9.4 REINFORCE

9.4.1 Main Idea

REINFORCE is the most basic form of policy gradient method. It's the vanilla policy gradient method.

9.4.2 Objective Function

REINFORCE uses following objective function;

$$J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G_t] = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right]. \quad (9.10)$$

We use some tricks to compute the gradient of $J(\pi_{\theta})$. Then we get

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} G_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right]. \quad (9.11)$$

Now we can use this to update the parameter θ . Since we cannot compute the exact expectation, we use Monte Carlo sampling to approximate $\nabla_{\theta} J(\pi_{\theta})$ as following:

$$\nabla_{\theta} J(\pi_{\theta}) \approx \sum_{t=0}^{T-1} G_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \quad (9.12)$$

9.4.3 Pseudocode

Algorithm 10: REINFORCE

Input : θ initial parameter
 γ discount rate
 α learning rate
Output: π_θ learned policy

```
1 initialize policy  $\pi_\theta$ 
2 repeat
3   generate episode  $\{s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T\}$  following  $\pi_\theta$ 
4   for  $t = 0, 1, \dots, T - 1$  do
5     // compute return-to-go
6      $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$ 
7     // policy update
8      $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$ 
9 until convergence
10 return  $\pi_\theta$ 
```

9.4.4 Implementation Notes

- When computing return-to-go, we can iterate backward from the terminal time to start, since $G_t \leftarrow \gamma G_{t+1} + r_{t+1}$ where $t = 0, 1, 2, \dots, T - 1$.

9.4.5 Pros and Cons

Pros:

- No need for model;
- Can cover both discrete and continuous actions;

Cons:

- Large variance;
- May converge to suboptimal policy;

Chapter 10

Actor-Critic Methods

10.1 Introduction

In this chapter, **actor-critic** algorithms are introduced. The actor refers to the policy update step, and critic refers to the value update step. The actor-critic methods are kind of policy gradient methods, with value function method mixed in. Instead of using the full return G_t to compute the loss, which has big variance, actor-critic uses value function for the estimate, and this value function is called **critic**.

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \log \pi(a_t | s_t, \theta_t) q_t(s_t, a_t) \quad (10.1)$$

- If $q_t(s_t, a_t)$ is estimated by the G_t , the corresponding algorithm is the REINFORCE [10](#).
- If $q_t(s_t, a_t)$ is estimated by using the TD algorithm, the corresponding algorithms are usually called **actor-critic**.

From here, the most simple actor-critic algorithm can be derived, the **Q actor-critic** algorithm. We just use two networks, one for policy and one for value function.

Algorithm 11: QAC

Input : $\pi_{\theta}(a|s)$ the policy function with initial parameter θ_0
 $q(s, a, w)$ the value function with initial parameter w_0

Output: π^* (sub)optimal policy

```
1 pick an action  $a_0$  following  $\pi_{\theta_0}(a|s_0)$ 
2 foreach time step  $t$  do
3   take  $a_t$ 
4   observe  $r_{t+1}$  and  $s_{t+1}$ , and then generate  $a_{t+1}$  following  $\pi_{\theta_t}(a|s_{t+1})$ 
   // policy update (actor)
5    $\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta} \log \pi_{\theta_t}(a_t | s_t) q_{w_t}(s_t, a_t)$ 
   // value update (critic)
6    $w_{t+1} \leftarrow w_t + \alpha_w [r_{t+1} + \gamma q_{w_t}(s_{t+1}, a_{t+1}) - q_{w_t}(s_t, a_t)] \nabla_w q_w(s_t, a_t)$ 
7 return  $\pi$ 
```

10.2 Advantage Actor-Critic (A2C)

10.2.1 Main Idea

The core idea of A2C is to reduce the variance by introducing the baseline. The following equation enable us to use the baseline when estimating state values.

$$\mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(A|S) q_{\pi}(S, A)] = \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(A|S) (q_{\pi}(S, A) - b(S))] \quad (10.2)$$

where $b(S)$ is a scalar function of S .

Proof.

$$\mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \log \pi_{\theta} \log \pi(A|S) b(S)] = \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}(s)} \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) b(s) \quad (10.3)$$

$$= \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}(s)} \nabla_{\theta} \pi_{\theta}(a|s) b(s) \quad (10.4)$$

$$= \sum_{s \in \mathcal{S}} \eta(s) b(s) \sum_{a \in \mathcal{A}(s)} \nabla_{\theta} \pi_{\theta}(a|s) \quad (10.5)$$

$$= \sum_{s \in \mathcal{S}} \eta(s) b(s) \nabla_{\theta} \sum_{a \in \mathcal{A}(s)} \pi_{\theta}(a|s) \quad (10.6)$$

$$= \sum_{s \in \mathcal{S}} \eta(s) b(s) \nabla_{\theta} 1 = 0. \quad (10.7)$$

□

The baseline reduces the variance of estimation of gradients. In particular, let

$$X(S, A) = \nabla_{\theta} \log \pi_{\theta}(a|s) [q_{\pi}(S, A) - b(S)] \quad (10.8)$$

Then, the true gradient is $\mathbb{E}[X(S, A)]$ and we estimate it with samples $x \sim X$. To better estimate the gradient, it would be favorable to make the variance of X small. The theoretically optimal baseline that minimizes $\text{var}(X)$ is

$$b^*(s) = \frac{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \log \pi_{\theta}(A|s)\|^2 q_{\pi}(s, A)]}{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \log \pi_{\theta}(A|s)\|^2]} \quad (10.9)$$

proof can be found at [Zha25]. But it is too complex to be useful in practice. In practice, we use the suboptimal version without the logarithm term, which is actually a value function, interestingly.

$$b(s) = \mathbb{E}_{A \sim \pi} [q_{\pi}(s, A)] = v_{\pi}(s) \quad (10.10)$$

10.2.2 Pseudocode

Algorithm 12: Advantage Actor-Critic Algorithm

Input : π_θ a policy function with initial parameter θ_0
 $v_w(s)$ a value function with initial parameter w_0
Output: π^* (sub)optimal policy

```

1 foreach time step  $t$  do
2   generate and take an action  $a_t$  following  $\pi_\theta$ 
3   observe  $r_{t+1}$  and  $s_{t+1}$ .
   // advantage
4    $\delta_t \leftarrow r_{t+1} + \gamma v_{w_t}(s_{t+1}) - v_{w_t}(s_t)$ 
   // actor (policy update)
5    $\theta_{t+1} \leftarrow \theta_t + \alpha_\theta \delta_t \nabla_\theta \log \pi_{\theta_t}(a_t | s_t)$ 
   // critic (value update)
6    $w_{t+1} \leftarrow w_t + \alpha_w \delta_t \nabla_w v_{w_t}(s_t)$ 
7 return  $\pi$ 

```

Using the baseline $b(s) = v_\pi(s)$, we get the following update equation:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \log \pi_{\theta_t}(a_t | s_t) [q_t(s_t, a_t) - v(s_t)] \quad (10.11)$$

If $q_t(s_t, a_t)$ and $v_t(s_t)$ were estimated by Monte Carlo, the algorithm is called **REINFORCE with a baseline**. If estimated by TD learning, the algorithm 12 is called **A2C** or **TD actor-critic**. Note that algorithm 12 estimates $q_t(s_t, a_t) - v_t(s_t)$ with TD error, which is $r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t)$.

10.2.3 Implementation Details

We can use n-step update to balance between bias and variance of estimation.

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^n v_t(s_{n+1}) \quad (10.12)$$

To stop the policy from being almost deterministic too quickly, we can add **entropy bonus** at the loss function ([Mni+16], [WP91]). Specifically, the loss function is

$$\mathcal{L} = \frac{1}{N} \sum_{i=0}^{N-1} \left[-\log \pi_\theta(a_{t+i} | s_{t+i}) A_{t+i} + c_v (v_\phi(s_{t+i}) - G_{t+i})^2 - c_e \mathcal{H}(\pi_\theta(\cdot | s_{t+i})) \right] \quad (10.13)$$

where

$$\mathcal{H}(\pi(\cdot | s)) = - \int \pi(a | s) \log \pi(a | s) da \quad (10.14)$$

$$\mathcal{H} = - \sum_a \pi(a | s) \log \pi(a | s), \quad (10.15)$$

A and G are advantage and return estimate, respectively.

The actor and critic can share parameters, when they are using the encoder. They can use different encoders, each of their own, or share one encoder.

10.2.4 Pros and Cons

Pros:

- Requires only one networks, opposed to QAC;
- Variance is decreased, opposed to the original REINFORCE without baseline;
- Applicable to both discrete and continuous actions;

Cons:

- If the baseline is incorrect, it can be biased;
- Sample efficiency is low, since it is an on-policy method;
- Vulnerable to big, sudden change of parameters. **PPO** was designed to resolve this;

Chapter 11

Model-based Methods

Chapter 12

Comparisons of Algorithms

Part III

Appendices

Chapter 13

Probability Theory

13.1 Introduction

This section introduces a theory of measure-theoretic probability, based on [BZ07].

13.2 Measure-Theoretic Probability

13.2.1 Basic Definitions

Definition 13.2.1.1 (σ -field)

Let Ω be a non-empty set. A σ -field on Ω is a family of subsets of Ω such that

1. the empty set $\emptyset \in \mathcal{F}$;
2. if $A \in \mathcal{F}$ then $\Omega \setminus A \in \mathcal{F}$;
3. if $A_1, A_2, \dots$ is a sequence of sets in $\mathcal{F}$, then $A_1 \cup A_2 \cup \dots \in \mathcal{F}$

Definition 13.2.1.2 (Probability Measure)

Let $\mathcal{F}$ be a σ -field on Ω . A **probability measure** P is a function $P : \mathcal{F} \rightarrow [0, 1]$ such that

1. $P(\Omega) = 1$;
2. if $A_1, A_2, \dots$ are pairwise disjoint sets ($A_i \cap A_j = \emptyset$ for all $i \neq j$) belonging to $\mathcal{F}$, then

$$P(A_1 \cup A_2 \cup \dots) = P(A_1) + P(A_2) + \dots \quad (13.1)$$

The triple $(\Omega, \mathcal{F}, P)$ is called a **probability space**. The sets belonging to $\mathcal{F}$ are called **events**. An event A is said to occur **almost surely** (a.s.) whenever $P(A) = 1$.

Definition 13.2.1.3 (Random Variable)

Let $\mathcal{F}$ be a σ -field on Ω . Then a function $\xi : \Omega \rightarrow \mathbb{R}$ is said to be $\mathcal{F}$ -measurable if the following is satisfied:

$$\{\xi \in B\} \in \mathcal{F} \text{ for every Borel set } B \in \mathcal{B}(\mathbb{R}). \quad (13.2)$$

where **Borel sets** refers to the elements of $\mathcal{B}(\mathbb{R})$ which is the smallest σ -field containing all intervals in $\mathbb{R}$. If the triple $(\Omega, \mathcal{F}, P)$ is a probability space, then such a function ξ is called a **random variable**.

Remark 13.2.1.1 (Short-hand Notation for Events)

The set of events such as $\{\xi \in B\}$ represents the set $\{\omega \in \Omega | \xi(\omega) \in B\}$.

Definition 13.2.1.4 (σ -field Generated by a Random Variable)

- The σ -field $\sigma(\xi)$ generated by a random variable $\xi : \Omega \rightarrow \mathbb{R}$ consists of all sets of the form $\{\xi \in B\}$, where B is a Borel set in $\mathbb{R}$.
- The σ -field $\sigma\{\xi_i : i \in I\}$ generated by a family of random variables $\{\xi_i : i \in I\}$ is defined to be the smallest σ -field containing all events of the form $\{\xi \in B\}$, where B is a Borel set in $\mathbb{R}$ and $i \in I$.

Theorem 13.2.1.1 (Doob-Dynkin Theorem)

Let ξ be a random variable. Then each $\sigma(\xi)$ -measurable random variable η can be written as

$$\eta = f(\xi) \quad (13.3)$$

for some Borel function $f : \mathbb{R} \rightarrow \mathbb{R}$, whose inverse image $f^{-1}(B)$ of any Borel set is again a Borel set. The converse also holds.

Definition 13.2.1.5 (Distribution of a Random Variable)

Every random variable $\xi : \Omega \rightarrow \mathbb{R}$ gives rise to a probability measure

$$P_\xi(B) = P\{\xi \in B\} \quad (13.4)$$

on $\mathbb{R}$ defined on the σ -field of Borel sets $B \in \mathcal{B}(\mathbb{R})$. We call P_ξ the **distribution** of ξ . The function $F_\xi : \mathbb{R} \rightarrow [0, 1]$ defined by

$$F_\xi(x) = P\{\xi \leq x\} \quad (13.5)$$

is called the **distribution function** of ξ .

These are the ones we've been using, before the measure-theoretic probability.

Chapter 14

Linear Algebra

14.1 Introduction

A short useful theorems of linear algebra.

14.2 Geshgorin Circle Theorem

Theorem 14.2.0.1 (Geshgorin Circle Theorem)

Bibliography

- [Axl20] Sheldon Jay Axler. *Measure, integration and real analysis*. Graduate texts in mathematics 282. Literaturverzeichnis: Seite 402. Cham, Switzerland: Springer Open, 2020. 411 pp. ISBN: 9783030331429.
- [BZ07] Zdzisław Brzeźniak and Tomasz Zastawniak. *Basic stochastic processes. A course through exercises*. 10. print. Springer undergraduate mathematics series. London: Springer, 2007. 225 pp. ISBN: 3540761756.
- [Gra20] Laura Graesser. *Foundations of deep reinforcement learning. Theory and practice in Python*. Ed. by Wah Loon Keng. Addison Wesley data & analytics series. Includes bibliographical references and index. Boston: Addison-Wesley, 2020. 379 pp. ISBN: 0135172381.
- [LS96] Michael L Littman and Csaba Szepesvári. “A generalized reinforcement-learning model: Convergence and applications”. In: *ICML*. Vol. 96. 1996, pp. 310–318.
- [Mni+13] Volodymyr Mnih et al. “Playing Atari with Deep Reinforcement Learning”. In: *CoRR* abs/1312.5602 (2013). DOI: [10.48550/arxiv.1312.5602](https://doi.org/10.48550/arxiv.1312.5602). arXiv: [1312.5602](https://arxiv.org/abs/1312.5602). URL: <http://arxiv.org/abs/1312.5602>.
- [Mni+16] Volodymyr Mnih et al. “Asynchronous Methods for Deep Reinforcement Learning”. In: *Proceedings of The 33rd International Conference on Machine Learning*. Ed. by Maria Florina Balcan and Kilian Q. Weinberger. Vol. 48. Proceedings of Machine Learning Research. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1928–1937. URL: <https://proceedings.mlr.press/v48/mniha16.html>.
- [Sut20] Richard S. Sutton. *Reinforcement learning. An introduction*. Ed. by Andrew Barto. Second edition. Adaptive computation and machine learning. Hier auch später erschienene, unveränderte Nachdrucke. Cambridge, Massachusetts: The MIT Press, 2020. 526 pp. ISBN: 9780262039246.
- [WP91] RONALD J. WILLIAMS and JING PENG. “Function Optimization using Connectionist Reinforcement Learning Algorithms”. In: *Connection Science* 3.3 (1991), pp. 241–268. DOI: [10.1080/09540099108946587](https://doi.org/10.1080/09540099108946587). eprint: <https://doi.org/10.1080/09540099108946587>. URL: <https://doi.org/10.1080/09540099108946587>.
- [Zha25] Shiyu Zhao. *Mathematical foundations of reinforcement learning*. [Tsinghua]: Tsinghua University Press, 2025. 1275 pp. ISBN: 9789819739448.